Learning Metamaterial Eigenmodes with Wavelet-Encoded Fourier Neural Operators

Han Zhang[1], Alexander Ogren[2], Cynthia Rudin[3],
Johann Guilleminot[1], L. Catherine Brinson[1]

[1]*Department of Mechanical Engineering and Materials Science, Duke University, Durham, NC, USA*
[2]*Department of Mechanical Engineering, California Institute of Technology, Pasadena, CA, USA*
[3]*Department of Electrical and Computer Engineering, Duke University, Durham, NC, USA*

Abstract

Machine learning surrogates based on neural operators have shown broad applicability in solving forward PDE problems. However, eigenvalue problems, in which an eigenparameter and one of several valid eigenmodes must be simultaneously solved, remain difficult because standard operator learning formulations assume a unique input–output map. This work demonstrates that Fourier Neural Operators (FNOs), combined with wavelet-based encodings of PDE inputs, can learn and inexpensively predict multiple eigenmodes of the elastic wave equation, corresponding to deformation modes of acoustic waves propagating through arbitrary metamaterial geometries. We provide theoretical motivation and experimental evidence for why wavelet encodings are well matched to the dual spatial–spectral structure of the FNO, enabling deterministic mode selection on both continuous and discretely valued geometries within a single model, and for why prediction accuracy varies with geometric discontinuities. These results carry broader implications for designing input encodings in other multi-mode PDE solvers based on spectral neural operators.

Keywords: Neural Operator, Metamaterials, Wavelets, Eigenmodes, Machine Learning

Table of Contents

1. Introduction

The numerical solution of partial differential equations (PDEs) underpins modern scientific computing and engineering design. Applications ranging from structural mechanics and fluid dynamics to wave propagation and materials engineering routinely rely on computationally intensive numerical methods such as finite element analysis (FEA) to obtain accurate solutions. While these methods are highly reliable, their computational cost can become prohibitive in settings that require repeated evaluations, such as optimization, uncertainty quantification, inverse design, and real-time control.

Recent advances in machine learning have introduced neural operators as a promising alternative to traditional numerical solvers. Unlike conventional neural networks that learn mappings between finite-dimensional vectors, neural operators learn mappings between function spaces, allowing them to approximate entire solution operators for families of PDEs. Models such as the Fourier Neural Operator (FNO) have demonstrated remarkable accuracy and computational efficiency across a variety of linear and nonlinear PDE problems while maintaining strong generalization capabilities across discretizations and geometries. These developments have generated significant interest in using neural operators as surrogate models for computational physics applications.

Despite this progress, existing neural operator architectures have primarily been developed and evaluated on PDEs whose governing parameters are known a priori and whose solutions are uniquely determined by the provided inputs. A large class of physically important problems, however, are governed by eigenvalue PDEs, in which both the solution field and one or more unknown PDE parameters must be determined simultaneously. Examples include vibration analysis, quantum mechanics, photonics, acoustics, and wave propagation in metamaterials. In these problems, multiple valid solutions may exist for the same physical domain, corresponding to different eigenvalue-eigenvector pairs. The ability to represent and distinguish among these multiple solutions is essential for accurately modeling the underlying physics.

Acoustic metamaterials provide a particularly compelling example of this chal-

lenge. These architected materials derive their properties from geometric structure rather than composition alone, enabling engineered interactions with propagating waves. Understanding and designing such systems often requires computing multiple deformation modes associated with different resonant frequencies and wave behaviors. Traditionally, these modes are obtained through computationally expensive eigenvalue analysis using finite element methods. A neural operator capable of accurately predicting multiple deformation modes directly from material geometry would significantly accelerate such iterative design and analysis processes.

However, extending neural operators to eigenvalue PDEs is nontrivial. Standard operator-learning formulations implicitly assume a one-to-one mapping between inputs and outputs, whereas eigenvalue problems admit multiple valid solutions for a given geometry. As a result, existing FNO architectures lack a mechanism for deterministically selecting among different eigenmodes. Furthermore, the representation of eigenvalue-related information must be compatible with the spectral architecture of the FNO, which processes information jointly in both spatial and frequency domains.

In this work, we introduce a framework that enables Fourier Neural Operators to solve eigenvalue PDEs associated with elastic wave propagation in acoustic metamaterials. The key idea is to augment the model inputs with wavelet-based encodings of PDE parameters that uniquely identify individual eigenmodes while remaining compatible with the FNO architecture. We demonstrate that these encodings allow a single neural operator to learn multiple valid solutions corresponding to distinct deformation modes and to reproduce those modes deterministically on demand. We further show that conventional spatial encodings and purely spectral encodings do not provide the same capability, highlighting the importance of the proposed wavelet representation. We also evaluate the same model on continuous- and discrete-valued geometries, recovering both displacement fields and eigenfrequencies, and examine how geometric discontinuities interact with the truncated spectral structure of the FNO.

The contributions of this paper are threefold:

1. We demonstrate that a single Fourier Neural Operator can learn multiple

eigenmodes across continuous and discrete metamaterial geometries, recovering both eigenvectors (displacement fields) and eigenvalues (eigenfrequencies).

2. We introduce wavelet encodings of modal parameters and explain theoretically why their spatial–spectral structure enables deterministic mode selection in FNOs.

3. We show that prediction accuracy degrades with geometric discontinuities and interpret this through spectral truncation, with implications for applying FNOs to problems with sharp interfaces.

Together, these results extend the applicability of neural operators beyond conventional forward PDE problems and establish a pathway toward efficient surrogate modeling of eigenvalue-driven physical systems, including acoustic metamaterials and other wave-based engineering applications.

2. Methodology

This section describes the methods used to generate data and train Fourier Neural Operators for acoustic metamaterial eigenmode prediction. We first summarize the FNO architecture, then define the metamaterial design space and the Bloch–FEA simulations used for supervised labels. We next introduce the wavelet encodings of modal parameters, describe dataset construction, and detail the training procedure.

2.1. Fourier Neural Operator

The principal theoretical motivation for Fourier Neural Operators is their universal approximation property over function spaces. Let $\mathcal{A}$ and $\mathcal{U}$ denote Banach spaces of input and output functions, respectively, and let

$$\mathcal{G} : \mathcal{A} \to \mathcal{U} \tag{1}$$

be a continuous nonlinear operator. It has been shown that, given sufficient latent width and Fourier modes, an FNO can approximate $\mathcal{G}$ arbitrarily well on compact subsets of $\mathcal{A}$. Unlike conventional neural networks, which approximate finite-dimensional

functions, FNOs approximate operators between infinite-dimensional function spaces by learning a sequence of global integral operators parameterized in the Fourier domain.

This theoretical framework assumes that both the input and output are continuous fields of infinite resolution. In practice, however, these fields are sampled on a finite computational grid. The continuous Fourier transform is therefore replaced by a discrete Fourier transform (DFT), and only a finite number of Fourier modes are retained during training. Consequently, the learned model approximates the continuous solution operator through its discrete representation while preserving the same spectral operator-learning architecture.

For the present work, the operator acts on discretized fields over a 32×32 pixel spatial grid. The input field consists of three channels containing encoded information for the metamaterial geometry, the stimulatory wavevectors, and the band number. The output field consists of five channels corresponding to the predicted eigenvalue, or frequency, and eigenvector, or displacement field, of the resulting deformation mode. Thus, the discretized learned operator takes the form

$$\mathcal{G}_\theta : \mathbb{R}^{32\times32\times3} \to \mathbb{R}^{32\times32\times5}. \tag{2}$$

The discretized FNO follows the standard lift–transform–project architecture:

1. **Lifting:** The input field is embedded into a higher-dimensional latent space through a pointwise lifting operator,

$$v_1(x) = \ell(a(x)), \qquad \ell : \mathbb{R}^3 \to \mathbb{R}^{d_L}. \tag{3}$$

2. **Fourier layers:** A sequence of Fourier layers is applied to the latent representation. Each layer updates the hidden field according to

$$v_{j+1}(x) = \sigma\left(\mathcal{F}^{-1}\left[R_j(k)\mathcal{F}[v_j](k)\right] + W_j v_j(x) + b_j\right), \tag{4}$$

where $\mathcal{F}$ denotes the discrete Fourier transform, $R_j(k)$ is the learnable spectral

kernel, W_j is a learnable pointwise linear transformation, b_j is a learnable bias term, and σ is the activation function. GELU was used in this work because it is smooth and differentiable.

3. **Projection:** The final latent representation is projected back to the desired output dimension through a pointwise projection operator,

$$u(x) = p(v_n(x)), \qquad p : \mathbb{R}^{d_L} \to \mathbb{R}^5. \tag{5}$$

FNOs are well suited for acoustic metamaterial simulation because the underlying physics is governed by spatially distributed wave equations, where the response at one location depends on the global geometry of the unit cell and the imposed wavevector. The Fourier layers are designed to use spectral convolutions to efficiently capture long-range interactions and periodic spatial structure, while the pointwise transformations preserve local geometric information. Although FNOs are commonly used as surrogate models for fixed PDE solution operators, the same operator-learning framework can be extended to eigenvalue problems by conditioning the input on modal parameters, such as the wavevector and band index, and training the network to output both the associated eigenvalue and eigenvector field. In this formulation, the FNO learns a parameterized eigensolution operator mapping geometry and modal encodings to the frequency and deformation mode of the acoustic unit cell.

This process is illustrated graphically in Figure 1. For our experiments, the NeuralOp Python package was used, which provides FNO model definitions with customizable numbers of layers, hidden channels, and input and output tensor sizes. The hyperparameter combinations explored, as well as the optimized final FNO configuration, are shown in Table S1 in Section 2.6. These hyperparameters are used alongside the fixed parameters of the problem space: model height 32, model width 32, input channels 3, and output channels 5.

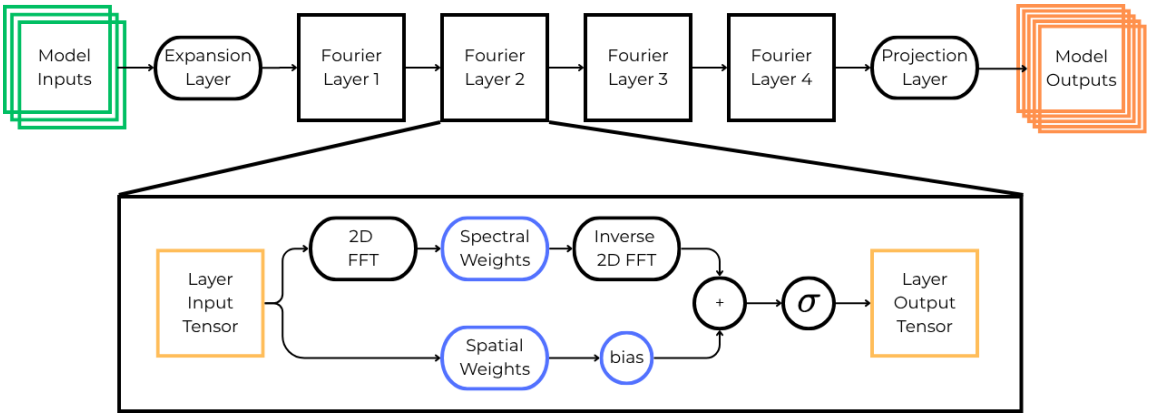


Figure 1: The FNO model architecture used in this project. The input data is first expanded according to the hidden-channels parameter, allowing multiple latent features to be represented between Fourier layers. Each Fourier layer is shown in detail in the expanded view. The data passing through a Fourier layer is processed through two learned branches: a pointwise branch, which applies a learned linear transformation in the spatial representation, and a spectral branch, which applies a fast Fourier transform (FFT), multiplies the retained Fourier modes by learned spectral weights, and then applies an inverse FFT. The outputs of both branches, along with a learnable bias term, are summed before passing through a nonlinear activation function. After passing through four Fourier layers, the latent representation is projected from the hidden-channel dimensionality to the final output dimensions.

2.2. Metamaterial Design Space

To train a surrogate model capable of predicting acoustic eigenmodes, a representative design space of acoustic metamaterials must first be defined. The design space was selected to provide sufficient geometric and material complexity to demonstrate operator learning for eigenvalue PDEs while maintaining computationally tractable finite element simulations for dataset generation.

The dataset is restricted to two-dimensional acoustic metamaterials composed of infinitely repeating square unit cells exhibiting eight-fold symmetry. Each unit cell is discretized onto a 32×32 spatial grid and represented by three material-property channels corresponding to the normalized elastic modulus, density, and Poisson's ratio. Material properties are normalized to the interval $[0, 1]$ to improve numerical conditioning during training, while the corresponding physical values are recovered using the fixed material constants listed in Table 1. The two constituent materials were chosen to represent a stiff steel alloy and a compliant polymer.

Parameter	E_{polymer}	E_{steel}	ρ_{polymer}	ρ_{steel}	ν_{polymer}	ν_{steel}
Units	Pa	Pa	kg/m^3	kg/m^3	–	–
Value	1×10^6	2×10^9	1200	8000	0.45	0.3

Table 1: Material parameters. The hard material is chosen to be a common steel alloy and the soft material a rubbery polymer.

Unit-cell geometries are synthesized by sampling a Gaussian process with a periodic covariance kernel and subsequently enforcing eight-fold symmetry. This procedure produces smoothly varying spatial material distributions while ensuring compatibility with periodically tiled square lattices. The sampled field assumes values between 0 and 1, where 0 represents pure polymer, 1 represents pure steel, and intermediate values denote an effective mixture of the two materials below the spatial resolution of the discretization.

To investigate the influence of geometry representation on model performance, two classes of unit cells were generated. The first consists of the continuous-valued Gaussian process realizations described above and is referred to as the *continuous geometry* dataset. The second is obtained by thresholding the Gaussian process samples to binary values, producing geometries composed exclusively of polymer and steel. This second collection is referred to as the *discrete geometry* dataset. Each representation comprises approximately one-half of the total dataset.

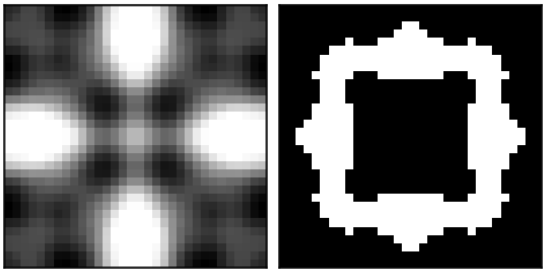


Figure 2: Example of a continuous geometry (left) and a discrete geometry (right) generated using the above process and parameters.

2.3. Wave Propagation Simulations

For each metamaterial geometry, wave propagation is simulated by solving the frequency-domain linear elastodynamic equation with finite element analysis (FEA). After spatial discretization and enforcement of Bloch periodicity, the governing problem takes the generalized eigenvalue form

$$\left(\mathbf{K}_r(\mathbf{k}) - \omega^2 \mathbf{M}_r(\mathbf{k})\right) \mathbf{u} = \mathbf{0}, \tag{6}$$

where ω is the angular eigenfrequency, $\mathbf{u}$ is the corresponding complex Bloch displacement eigenvector, and $\mathbf{k}$ is the wavevector in the irreducible Brillouin zone (IBZ). The reduced stiffness and mass matrices $\mathbf{K}_r(\mathbf{k})$ and $\mathbf{M}_r(\mathbf{k})$ are obtained from geometry-dependent global finite-element matrices $\mathbf{K}$ and $\mathbf{M}$ via a wavevector-dependent Bloch transformation matrix $\mathbf{T}(\mathbf{k})$,

$$\mathbf{K}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{K}\, \mathbf{T}(\mathbf{k}), \qquad \mathbf{M}_r(\mathbf{k}) = \mathbf{T}(\mathbf{k})^\dagger \mathbf{M}\, \mathbf{T}(\mathbf{k}), \tag{7}$$

with $(\cdot)^\dagger$ the Hermitian transpose. Here $\mathbf{T}(\mathbf{k})$ maps the full mesh degrees of freedom to an independent set while applying phase factors $\mathrm{e}^{i\mathbf{k}\cdot\mathbf{R}}$ across periodic faces of the unit cell; $\mathbf{K}$ and $\mathbf{M}$ themselves depend only on the material field and are assembled once per geometry. These intermediate matrices are described briefly so the reader can compare the explicit FEA pipeline (Figure 3) with the Fourier neural operator architecture in Section 2.1; they are not inputs to the neural operator.

The IBZ is discretized into 25 x-direction and 13 y-direction wave numbers, totaling $25 \times 13 = 325$ unique wavevectors covering the IBZ half-plane. For each wavevector, the six lowest eigenfrequencies (bands) were computed using a custom FEA MATLAB script (see Section 4 for implementation details).

Each eigenvector $\mathbf{u}$ consists of two complex-valued displacement fields, (u_x, u_y), which are separated into their real and imaginary components to produce four real-valued output channels, $(u_{x,real}, u_{x,imag}, u_{y,real}, u_{y,imag})$. The corresponding eigenfrequency ω is encoded as a spatially uniform fifth channel, resulting in a target tensor of size $5 \times 32 \times 32$. Combined with the three-channel material representation

described in the previous subsection, each simulation therefore produces a supervised learning sample mapping

$$\mathbb{R}^{3\times32\times32} \rightarrow \mathbb{R}^{5\times32\times32}.$$

The overall finite element data-generation procedure is summarized in Figure 3.

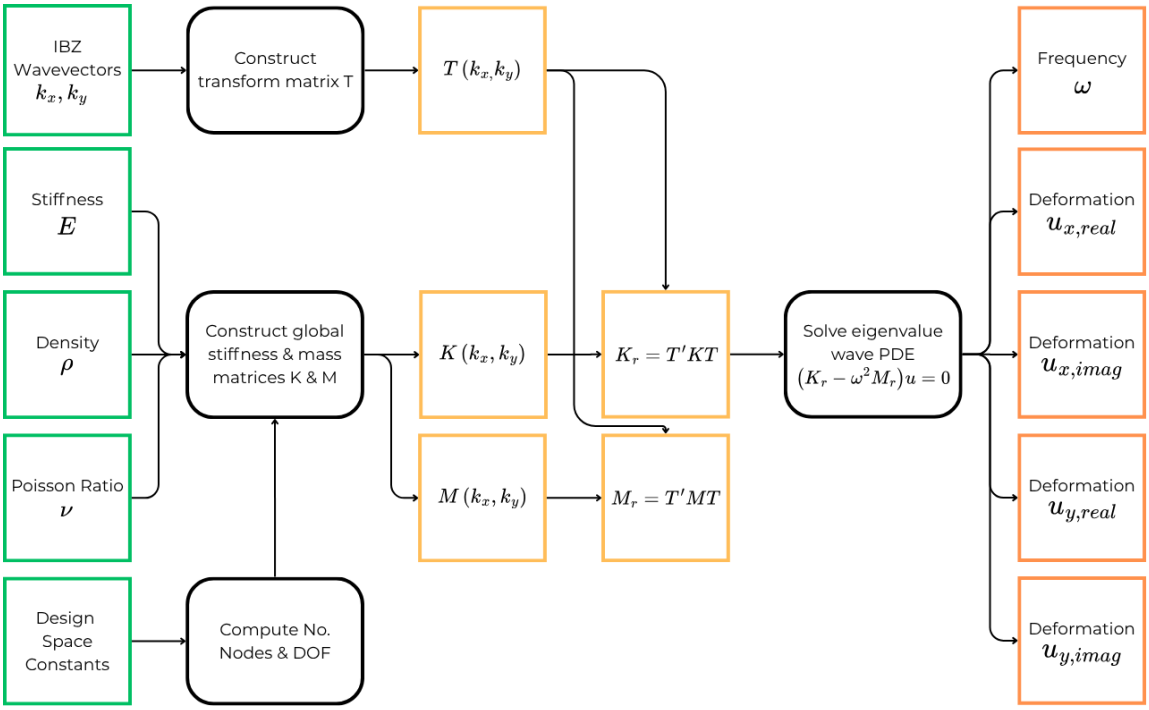


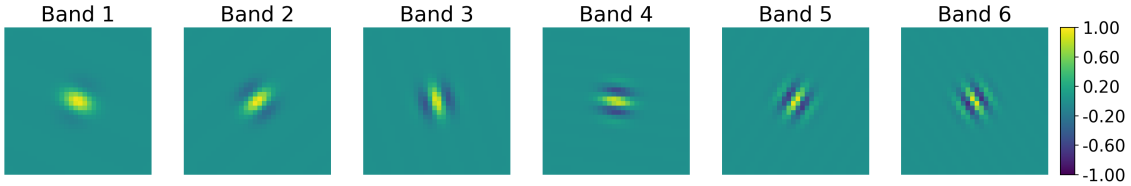
Figure 3: Flowchart of the FEA data generation method. Green squares represent varying inputs to the process, and orange squares represent computed quantities of the FEA process. Black rounded rectangles represent subroutines of the FEA process. The process starts with inputs of defined densities, stiffnesses, and Poisson ratios for each pixel of a metamaterial unit cell geometry (each a 32×32 pixel grid of values ranging from 0 to 1), along with a chosen k_x and k_y wavevector representing the stimulus to the metamaterial. A custom optimized FEA solver takes these inputs and produces intermediate matrices $(\mathbf{K}, \mathbf{M}, \mathbf{T})$, forms the reduced operators $(\mathbf{K}_r, \mathbf{M}_r)$, and then solves Eq. (6) for the displacement fields (eigenvectors) and frequencies ω (eigenvalues) for the given inputs.

2.4. Input Wavelet Encoding

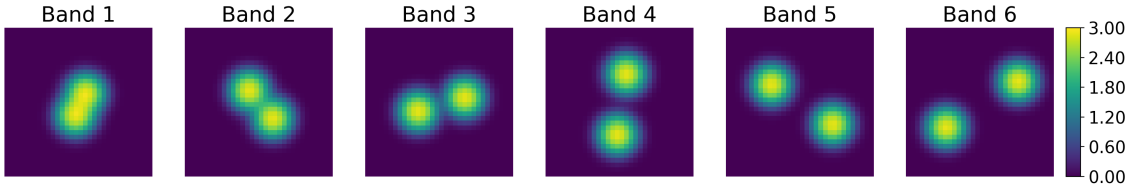
To allow the FNO model to toggle between PDE solutions for a given geometry, information must be fed to the model indicating which excitation waveform (wavevectors) and deformation mode (band) we are looking for a solution to. This

information is given in the form of a wavevector embedding, which we have found to yield superior results compared with a constant field embedding (2D matrices of the same value) or a sinusoidal embedding (2D sinusoidal fields with constants encoded as the sinusoidal frequency). A discussion on why wavelet encodings are superior is found in Section 3.4.

For the wavelet encodings, we use a 1D Gabor wavelet transform to map band numbers to unique wavelet images (see Algorithm 1), and a 2D Gabor wavelet transform to map x and y wavevectors to another set of unique wavelet images (see Algorithm 2). These representations show rich variation both in the spatial domain and in the Fourier domain, which allows for FNO model weights of both branches to richly interact with the wavelet representation, leading to better learning outcomes.

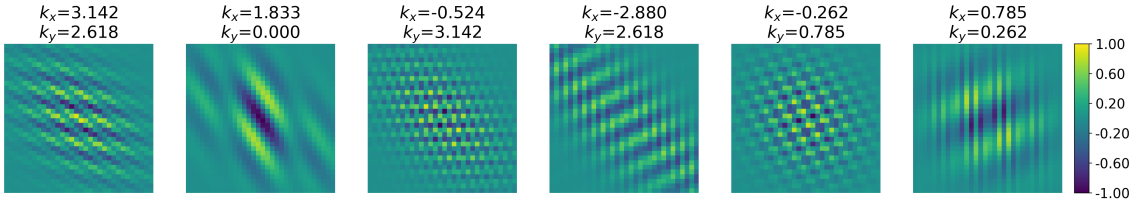


(a) The wavelet spatial encoding for band integer.

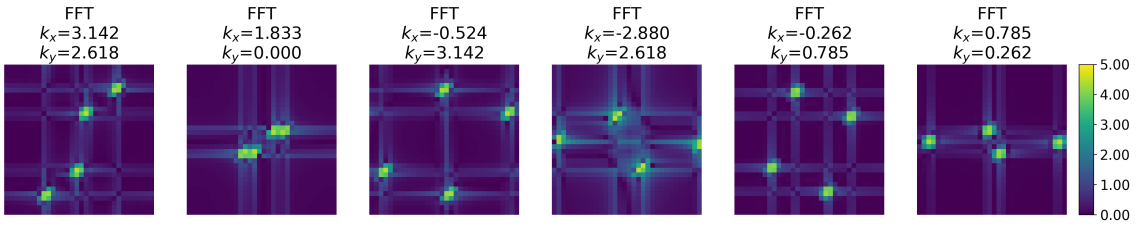


(b) The FFT of the above wavelet spatial encodings.

Figure 4: The wavelet encoding of the band integer (Fig. 4a) and its FFT (Fig. 4b). Each band corresponds to a unique wavelet with representation in both spatial and spectral domains.



(a) The wavelet spatial encoding for x and y wavevectors.



(b) The FFT of the above wavelet spatial encodings.

Figure 5: The wavelet encoding of the x and y wavevectors (Fig. 5a) and its FFT (Fig. 5b). The physical wavevectors are shown in the subplot titles. For each (k_x, k_y) pair there is a unique mapping in both the spatial and spectral domains.

When encoding continuous constants with oscillatory functions on a finite grid, it is important to verify that distinct parameter values do not alias into nearly identical patches. Otherwise the model cannot distinguish neighboring wavevectors, and mode selection collapses. We therefore checked pairwise cosine similarity of the mean-centered $\log_{10}|\text{FFT}|$ spectra of all 325 IBZ wavevector encodings produced by Algorithm 2 (Supplementary Information). The resulting similarity matrix is shown in Figure 6; the maximum off-diagonal similarity is 0.862, well below near-duplicate levels, confirming that the chosen encoding constants introduce no appreciable aliasing across the IBZ grid.

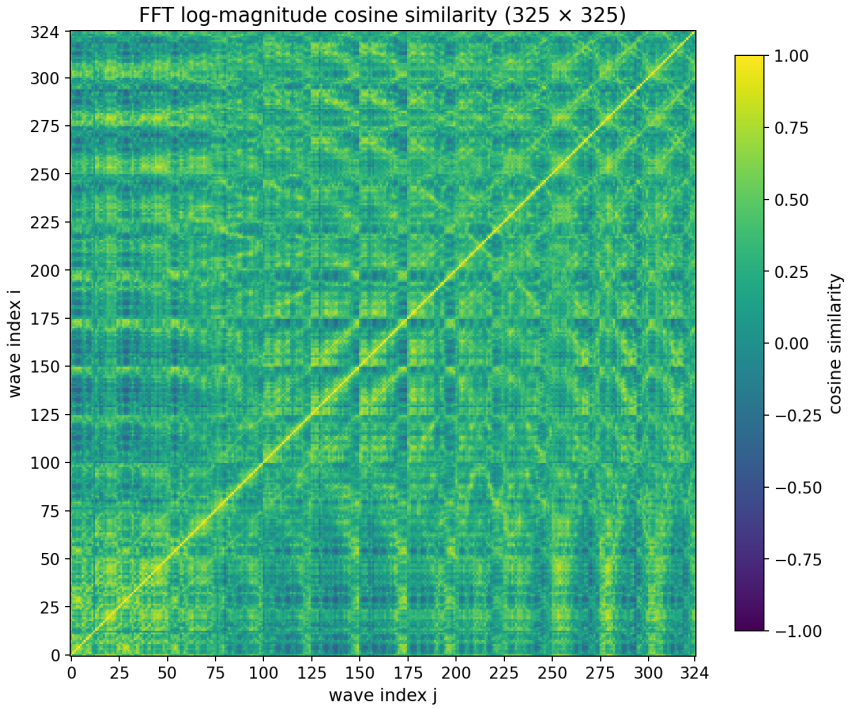
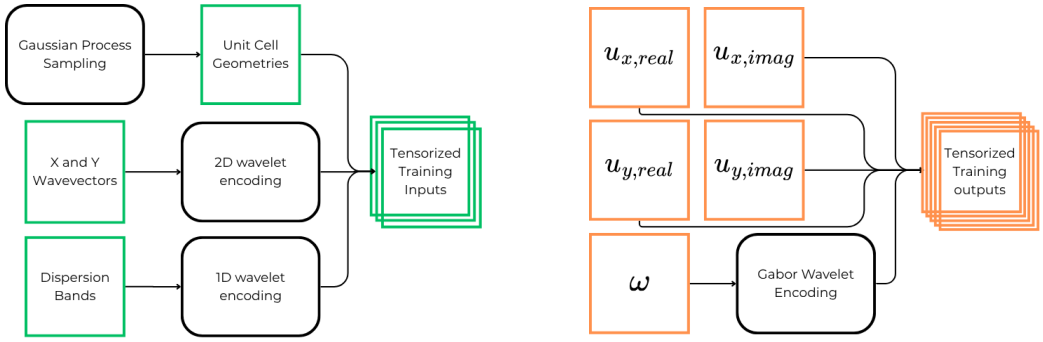


Figure 6: Pairwise cosine similarity of mean-centered $\log_{10}|\mathrm{FFT}|$ spectra for the 325 IBZ wavevector encodings generated by Algorithm 2. The maximum off-diagonal similarity is 0.862, indicating that distinct (k_x, k_y) pairs remain spectrally separable on the 32×32 grid.

2.5. Dataset Construction

Our full dataset contains 24,000 geometries, randomly generated as described in prior subsections. Half are discrete-valued, with only 0s or 1s in each pixel location representing one of the two materials, while the other half are continuous-valued, with any value in $[0, 1]$ at each pixel representing a blended material. Each geometry has six bands and 325 wavevectors per band (a mesh grid of the IBZ half-plane), resulting in $24000 \times 6 \times 325 = 46{,}800{,}000$ sample pairs of inputs with shape $(3 \times 32 \times 32)$ and outputs of shape $(5 \times 32 \times 32)$. The data are stored as float16 PyTorch tensors.



(a) Construction of the input tensors

(b) Construction of the output tensors

Figure 7: Subfigures a and b show the processes by which input data from Section 2.2 and output data from Section 2.3 are tensorized for ingestion by the FNO model. Note the wavelet embedding steps applied to fundamentally scalar quantities.

2.6. Training Procedure

The Fourier Neural Operator was trained using supervised learning on paired inputs and outputs described in Section 2. Each training sample consists of a $3\times32\times32$ input tensor (geometry and wavelet embeddings) and a $5 \times 32 \times 32$ output tensor (displacement field components and eigenfrequency).

Six loss functions were evaluated as training objectives: mean absolute error (MAE), mean squared error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural similarity index (SSIM). Losses were aggregated across all five output channels. Huber loss yielded the best overall performance and was used for all reported results; its definition is given below. Details on the other loss functions are provided in the Supplementary Information.

$$\mathcal{L}_{\text{Huber}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^{4} \begin{cases} \frac{1}{2} \left(\hat{u}_c(i,j) - u_c(i,j) \right)^2, & |\hat{u}_c(i,j) - u_c(i,j)| \leq \delta, \\ \delta \left(|\hat{u}_c(i,j) - u_c(i,j)| - \frac{\delta}{2} \right), & |\hat{u}_c(i,j) - u_c(i,j)| > \delta. \end{cases}$$

where $H = W = 32$ (pixels) and $c \in \{0, 1, 2, 3, 4\}$ corresponds to the 5 output channels: eigenfrequency, followed by real and imaginary components of x and y displacements.

For training the FNO model, combinations of model layers, hidden channels, activation function, loss function, optimizer, and training hyperparameters were evaluated. The AdamW optimizer achieved the best performance compared with Adam, other momentum-based optimizers, and SGD. The candidate settings and selected values are reported in Table S1 in Section S1, with the selected values italicized.

The selected architecture uses four Fourier layers, 128 hidden channels, and GELU activations. Increasing the depth or width beyond these values did not appreciably improve validation performance and increased both training cost and the tendency to overfit. The selected training configuration uses Huber loss throughout, AdamW optimization, a learning rate of 2×10^{-3}, a batch size of 512, and 12 epochs. All results reported in this article were generated using this configuration.

Final model performance did not appreciably change with modest variations in scheduler step size or batch size, so these parameters were fixed at 4 and 512, respectively, for the remaining training runs. Learning rate and time to convergence were inversely related: higher learning rates accelerated convergence but produced less stable optimization. In particular, learning rates above approximately 10^{-2} occasionally caused an abrupt, irreversible increase in loss, after which training plateaued. We therefore selected a learning rate of 2×10^{-3}, which provided the best balance between convergence speed and stability.

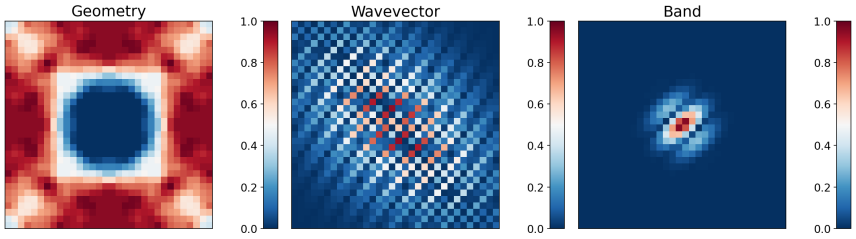
3. Results and Discussion

3.1. Displacement Field Prediction

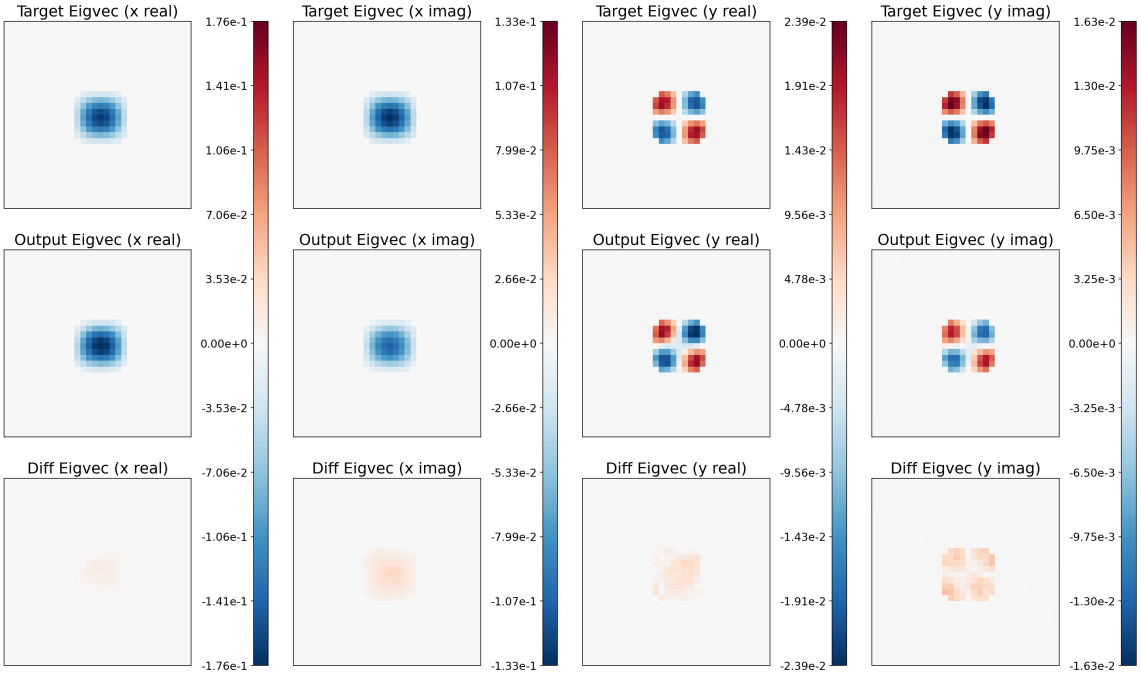
We next evaluate the trained Fourier Neural Operator on its predictions of the Bloch displacement fields (u_x, u_y). For each unseen test sample, the prediction error is measured by the normalized mean absolute error (NMAE) over the four real-valued displacement channels, $(u_{x,\mathrm{real}}, u_{x,\mathrm{imag}}, u_{y,\mathrm{real}}, u_{y,\mathrm{imag}})$. Samples are ranked by performance, defined inversely with NMAE, so larger losses correspond to lower performance percentiles. We select representative cases at the 25th, 50th, and 75th performance percentiles, corresponding to relatively weak, median, and relatively

strong predictions. The same percentile comparison is shown for both the binarized and continuous material datasets, allowing direct visual assessment of how well the model recovers the spatial structure of the displacement modes across contrasting geometry regimes.

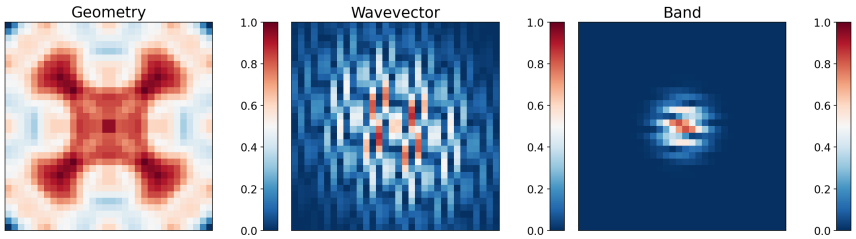
3.1.1. Continuous Geometries



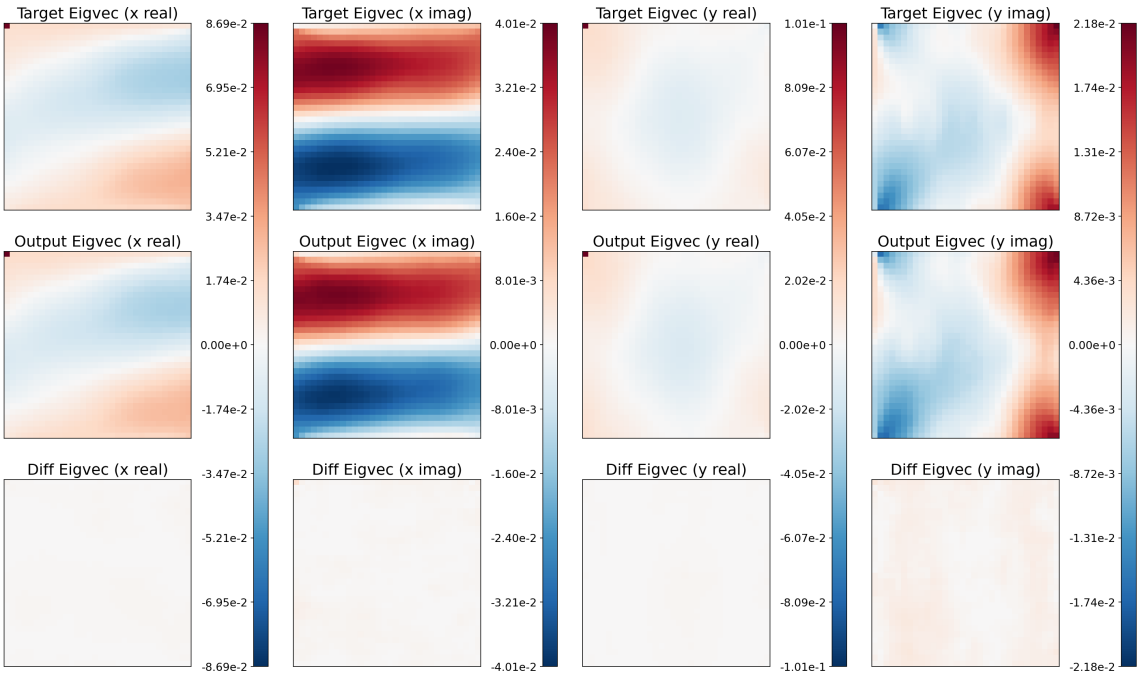
(a) Input sample at the 25th percentile of performance for unseen continuous-valued geometries.



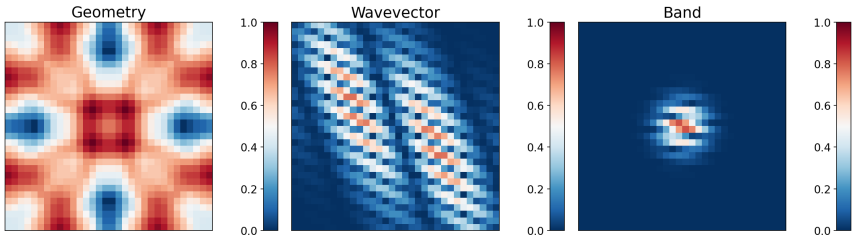
(b) Output predictions at the 25th percentile of performance for unseen continuous-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. At the 25th percentile, prediction outcomes are already fairly good, with accurate representations of scale and structure.



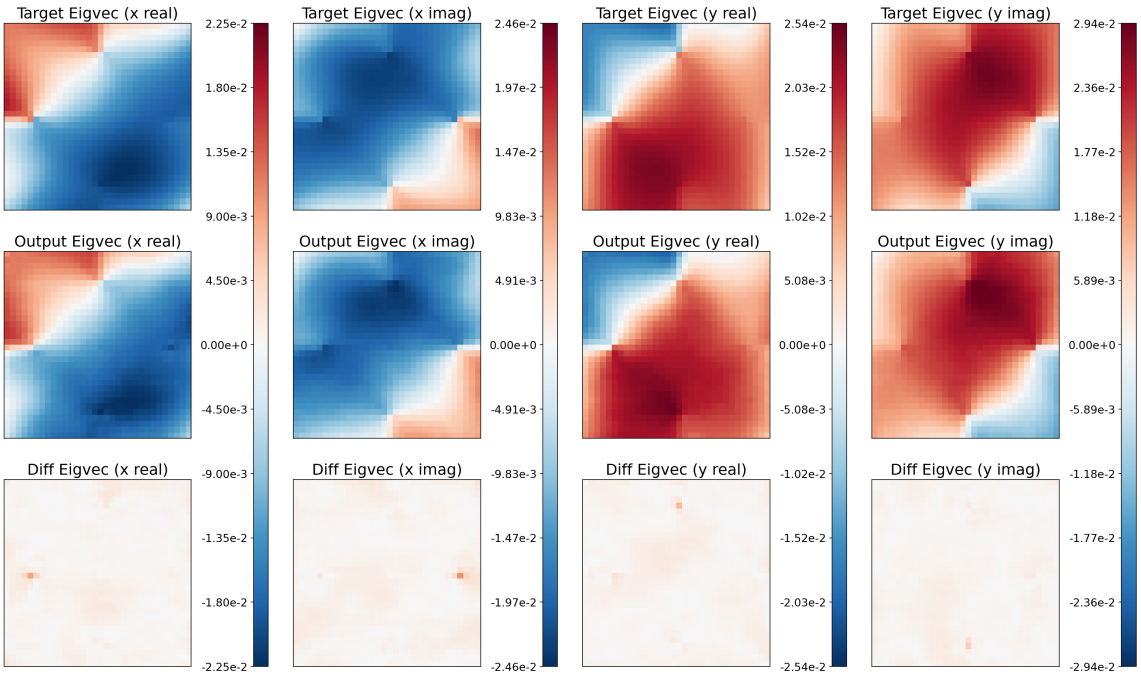
(a) Input sample at the 50th percentile of performance for unseen continuous-valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen continuous-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. Predictions are able to consistently capture complex structures without graininess.



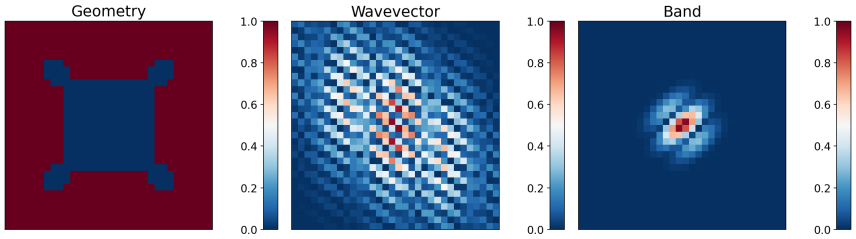
(a) Input sample at the 75th percentile of performance for unseen continuous-valued geometries.



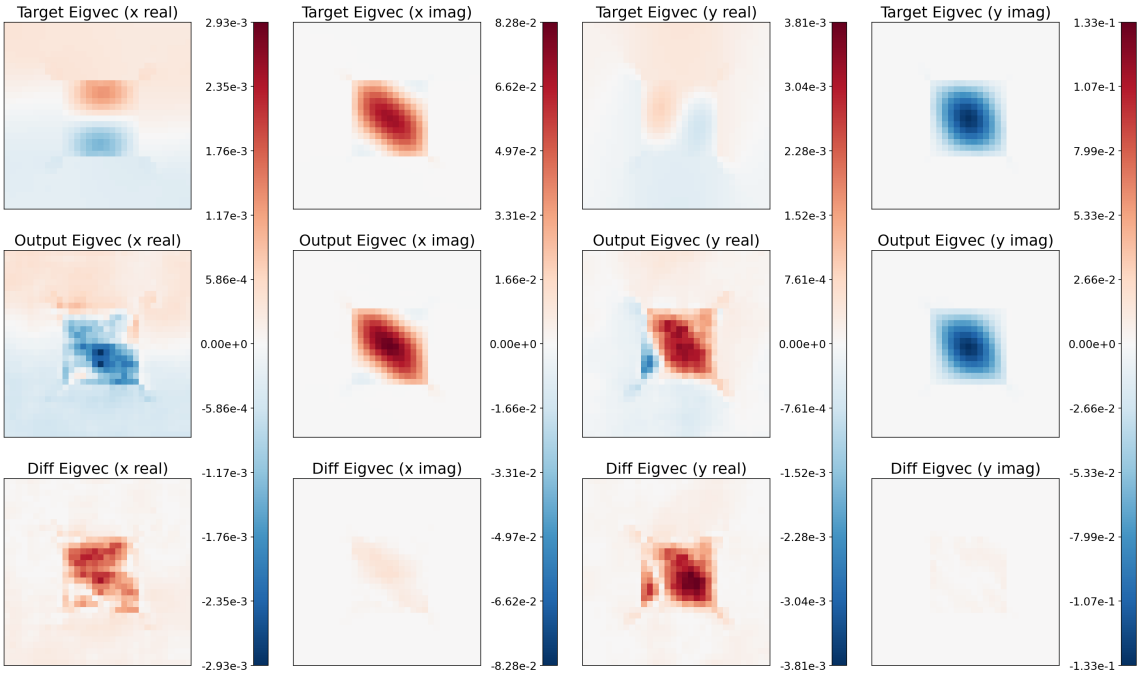
(b) Output predictions at the 75th percentile of performance for unseen continuous-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. At this point, the target and prediction are virtually indistinguishable by eye.

These results demonstrate that with wavelet encodings the FNO performs well on continuous geometries, faithfully reproducing the displacement fields in most cases. Model predictions in the upper performance quartiles match the targets well in structure and scale, with average pixel relative errors decreasing smoothly from on the order of 10^{-2} to 10^{-4} as performance percentile increases (see Figure 14a for detailed statistics).

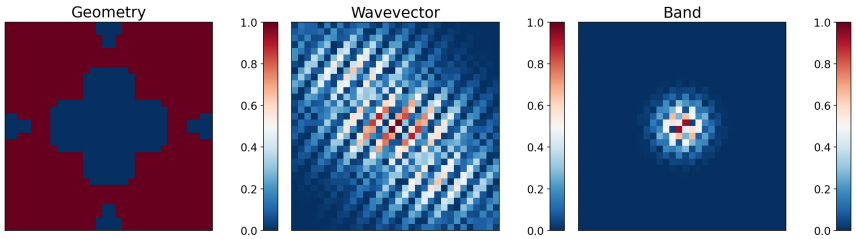
3.1.2. *Discrete Geometries*



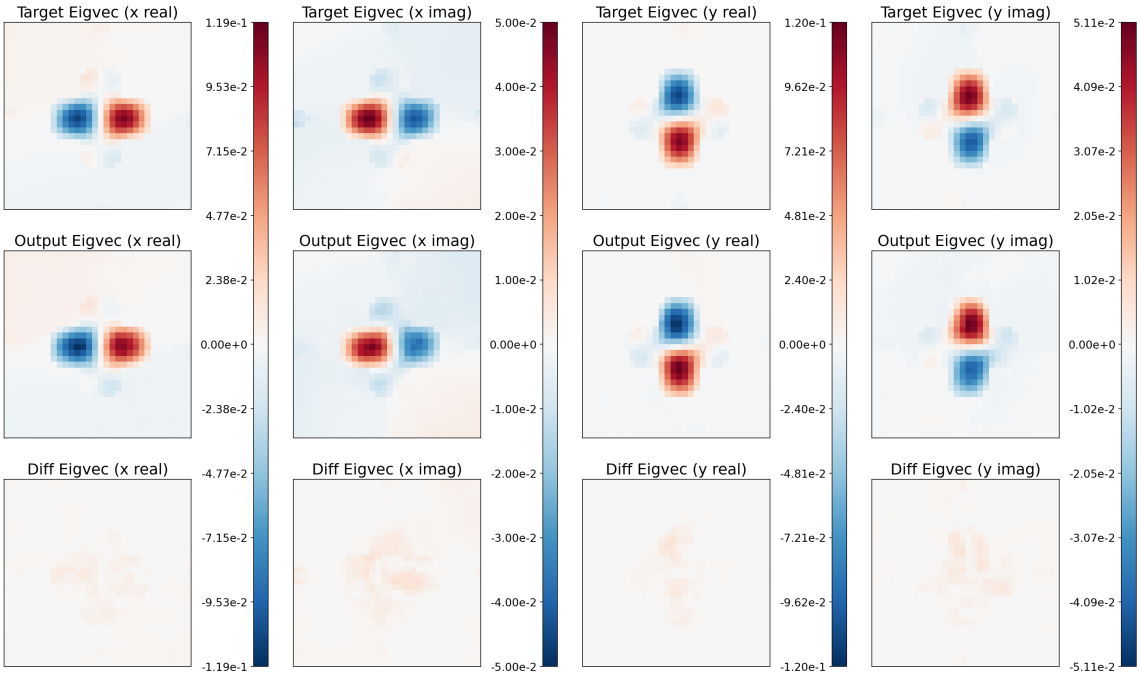
(a) Input sample at the 25th percentile of performance for unseen discrete-valued geometries.



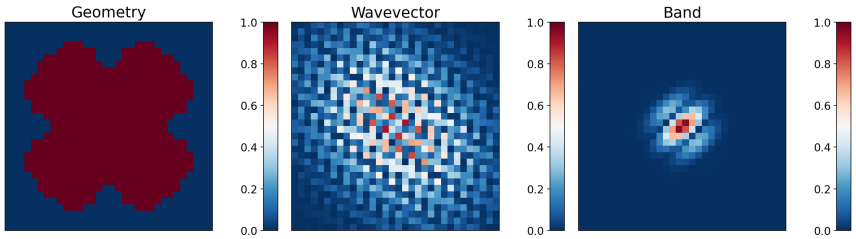
(b) Output predictions at the 25th percentile of performance for unseen discrete-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. Because Huber loss penalizes absolute residuals, for this sample the model prioritizes the two imaginary displacement channels, which are one to two orders of magnitude larger than the real channels. Absolute error is therefore low overall, but relative error remains large on the smaller-magnitude channels. (Note that the colorbars for each column are independently scaled.)



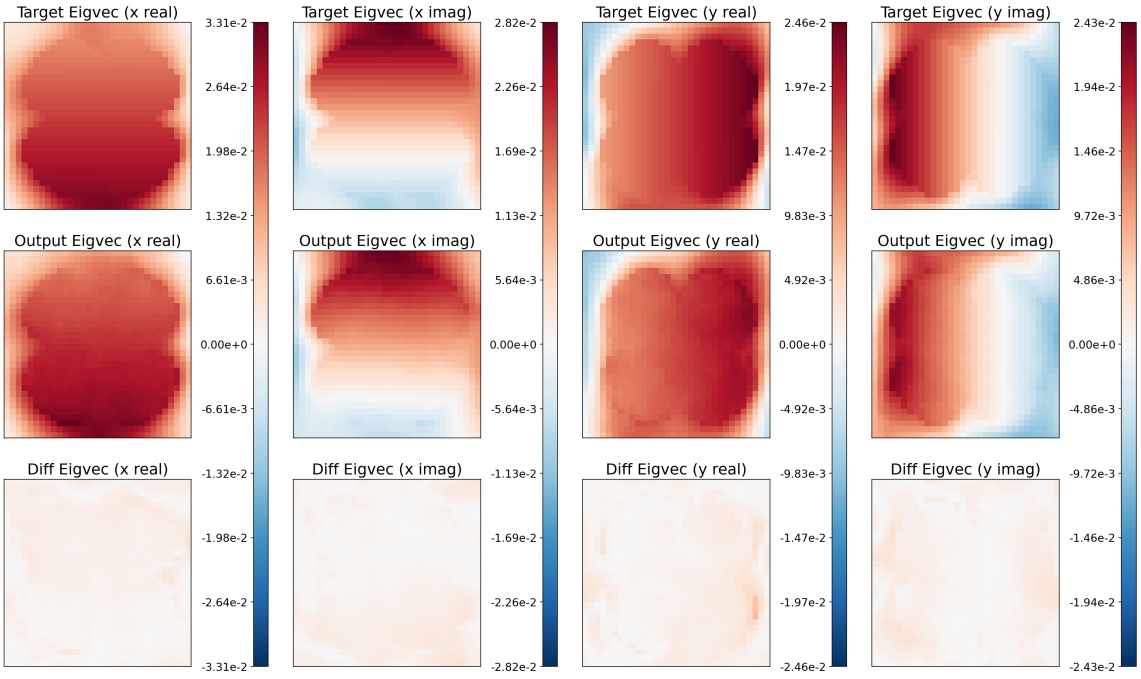
(a) Input sample at the 50th percentile of performance for unseen discrete-valued geometries.



(b) Output predictions at the 50th percentile of performance for unseen discrete-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. Absolute and relative errors are low, with strong agreement in structure and scale between targets and predictions.



(a) Input sample at the 75th percentile of performance for unseen discrete-valued geometries.



(b) Output predictions at the 75th percentile of performance for unseen discrete-valued geometries. The first row shows target fields, the second row shows predictions, and the third row shows difference fields. At this point, disagreements between targets and predictions become very hard to see by eye.

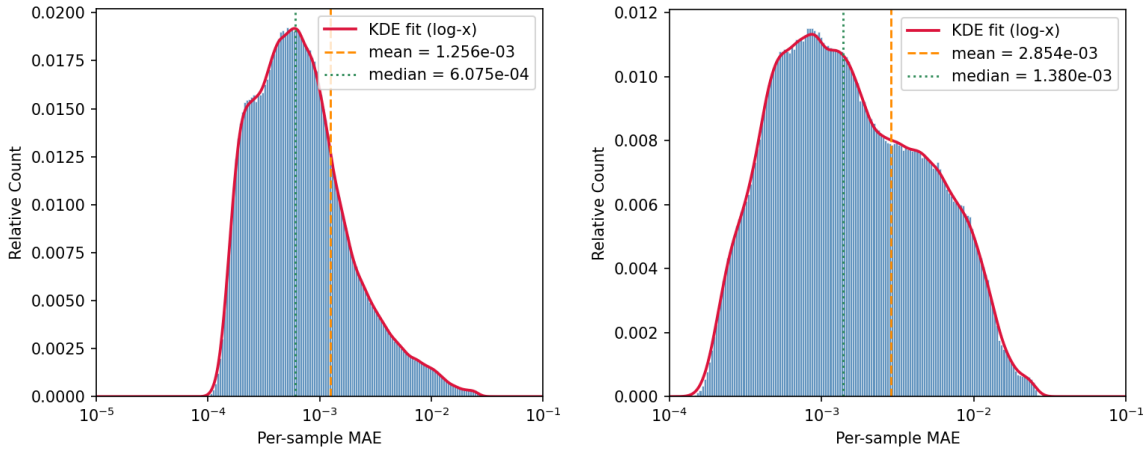
As in the continuous-geometry case, these results demonstrate that with wavelet encodings the FNO performs well on discrete geometries, faithfully reproducing the displacement fields in most cases. Model predictions in the upper performance quartiles match the targets well in structure and scale, with average pixel relative errors decreasing smoothly from on the order of 10^{-2} to 10^{-4} as performance percentile increases (see Figure 14b for detailed statistics).

3.1.3. Continuous Versus Discrete Performance

That a single model predicts well on both continuous and discrete geometries is a nontrivial result for FNOs paired with wavelet encodings. Prior encoding strategies based on constant fields and sinusoidal fields performed far worse, producing grainy structures or mode collapse, because conditioning information for the wavevector and band could not propagate effectively through the spectral branch of the FNO layers. Performance over all unseen samples in the test set is shown in Figure 14.

The poorer performance on binarized geometries is consistent with basic operator-learning theory. Continuous material fields lie in smoother function classes, whereas binary two-phase designs contain jump discontinuities with nonexistent derivatives at material interfaces. Since an FNO approximates the FEA operator $\mathcal{G} : \mathcal{U} \to \mathcal{V}$ through truncated Fourier representations, it is naturally advantaged when both inputs and outputs are comparatively smooth. Discrete geometries thus perform worse due to sharp boundaries, localized gradients, and broadband high-wavenumber content that are poorly captured on a fixed 32×32 grid and are mismatched to smooth wavelet encodings. Additionally, continuous geometries more often contain large regions of near-zero displacement, which inflate relative error through a small denominator.

These results indicate two important takeaways. The first is that FNOs with wavelet encodings can learn multiple deformation modes for each geometry, which the user can select by setting the wavevector and band inputs. The second is that the same model can learn across continuous and discrete geometry distributions, suggesting that some aspects of the underlying eigenvalue problem may be captured in a geometry-agnostic way. Exploring that possibility more systematically is left for future work.

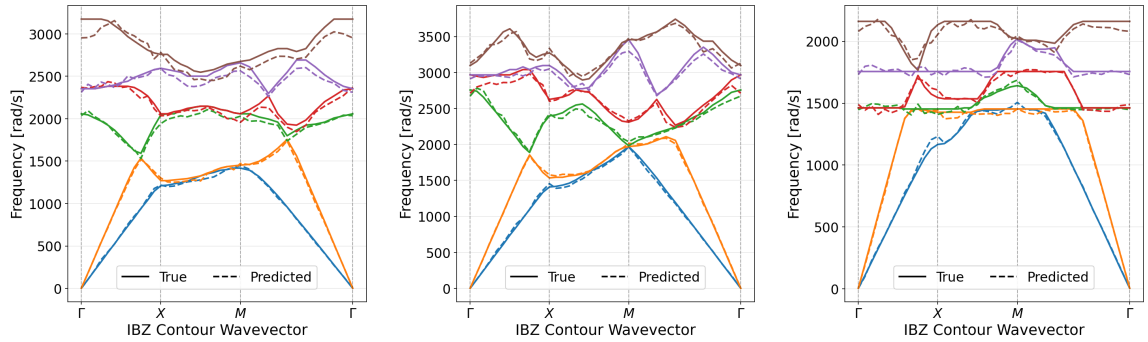


(a) Performance histogram for continuous geometries.

(b) Performance histogram for discrete geometries.

Figure 14: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4}, indicating that the same model is able to learn continuous and discrete geometry cases.

3.2. Dispersion Band Reconstruction



(a) 25th percentile performance.

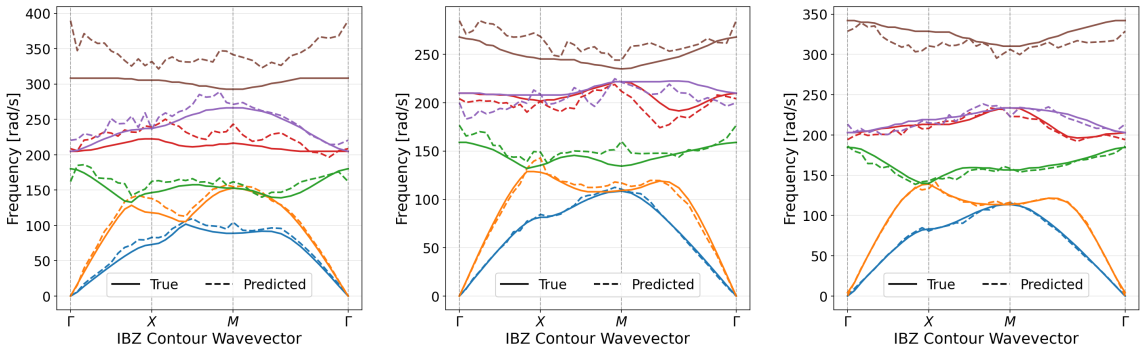
(b) 50th percentile performance.

(c) 75th percentile performance.

Figure 15: Predicted and ground-truth dispersion bands for representative unseen continuous-valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

These plots show eigenfrequency predictions across all 325 wavevectors of the IBZ half-plane for continuous-valued geometries, restricted to the horizontal, vertical, and diagonal IBZ traversals. Because the eigenfrequency is encoded with a logarithmic

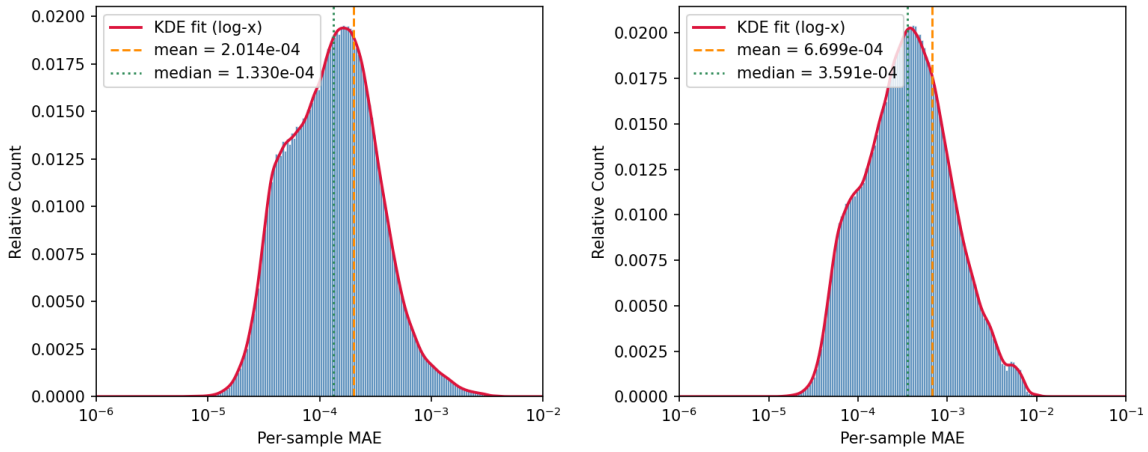
transform to support multi-scale learning, errors tend to scale with the target magnitude, and higher bands can appear more deviant on a linear plot. Overall, the model achieves an average prediction error below 1% on the encoded eigenfrequency across unseen samples.



(a) 25th percentile performance. (b) 50th percentile performance. (c) 75th percentile performance.

Figure 16: Predicted and ground-truth dispersion bands for representative unseen discrete-valued geometries spanning the 25th, 50th, and 75th percentiles of prediction performance.

As in the continuous-geometry figures, these plots show eigenfrequency predictions across all 325 wavevectors of the IBZ half-plane for discrete-valued geometries, restricted to the horizontal, vertical, and diagonal IBZ traversals. Overall, the model again achieves an average prediction error below 1% on the encoded eigenfrequency across unseen samples.



(a) Performance histogram for continuous geometries.

(b) Performance histogram for discrete geometries.

Figure 17: Comparison of relative prediction error distributions for continuous-valued and discrete metamaterial geometries for the same model. While there are some tail outliers, most samples fall within the range of 10^{-2} to 10^{-4}, indicating that the same model is able to learn continuous and discrete geometry cases.

3.3. Dependence of Prediction Error on Band and Boundary Length

The preceding subsections showed that, although the same wavelet-conditioned FNO can predict displacement fields for both continuous and discrete unit cells, discrete geometries are systematically more difficult, with absolute and relative errors larger (Figures 14, 17), and the harder quartiles of the discrete test set degrading more visibly than their continuous counterparts. This gap is expected from the architecture of the FNO itself. Each Fourier layer retains only a truncated set of spectral modes after the discrete FFT. Smooth continuous material fields, and the comparatively smooth displacement modes they induce, concentrate their energy in a modest number of low-to-moderate wavenumbers and are therefore well represented by that truncated Fourier basis. Binary geometries, by contrast, contain jump discontinuities at material interfaces. Under an FFT, such discontinuities produce a broadband spectrum whose coefficients decay slowly with wavenumber, so a substantial fraction of the high-frequency content needed to resolve sharp phase boundaries lies outside the retained modes. The spectral branch therefore sees a degraded, band-limited version of the interface, yielding Gibbs-type ringing or blur and a harder operator-learning problem

precisely when geometric discontinuity is more severe. If this spectral-truncation explanation is correct, then even among discrete geometries the prediction error should increase with the boundary interface length. To test this empirically, we define the *boundary length* of each binarized unit cell as the number of interior four-connected pixel edges separating a 0-valued pixel from a 1-valued pixel. This scalar is a discrete measure of interface complexity. For each of the 1000 unseen discrete test geometries, we compute the mean absolute error (MAE) of the predicted displacement channels, averaged over all 325 IBZ wavevectors, separately for each of the six retained eigenbands. The resulting relationships are shown in Figure 18.

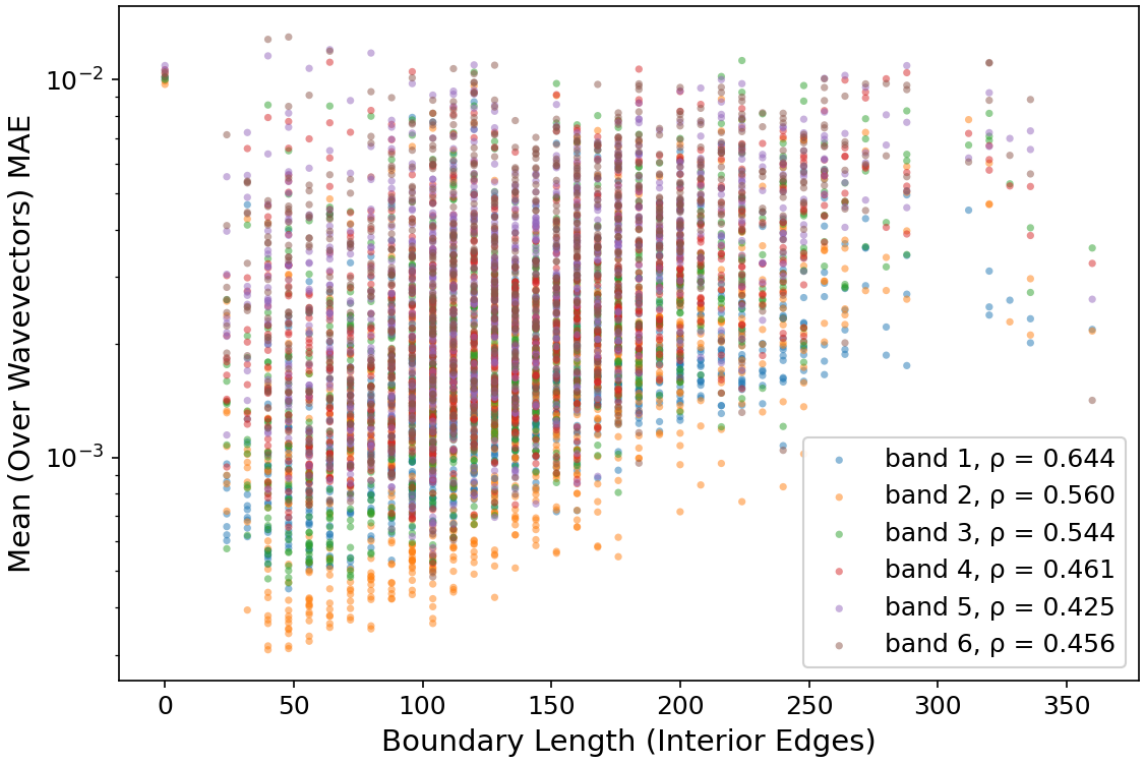


Figure 18: Mean displacement MAE versus interface boundary length, separated by color for each of the six eigenbands on the discrete test set. Points correspond to average performance on unit-cell geometries. Legend values ρ are Spearman rank correlations. A positive trend appears for every band, consistent with degraded FNO accuracy as the total length of 0/1 material interfaces increases.

Two complementary trends are apparent. First, for every band there is a positive

monotonic association between boundary length and MAE, with Spearman correlations ranging from $\rho = 0.644$ for band 1 to $\rho \approx 0.42$–0.56 for the higher bands. Geometries with short interfaces are concentrated at lower absolute error, while geometries with long, meandering phase boundaries systematically occupy the upper portion of the cloud. This supports the interpretation that the hard cases for the learned operator are those for which the truncated Fourier representation must resolve more extensive sharp material transitions on the fixed 32×32 grid. Second, absolute error itself increases with band index. Averaged over the test geometries, the mean wavevector-averaged MAE rises from approximately 1.6×10^{-3} for band 1 to approximately 3.8×10^{-3} for band 6. Higher bands correspond to more rapidly oscillating Bloch modes that concentrate energy nearer material interfaces and therefore inherit more of the geometric difficulty encoded by boundary length. Together with the continuous-versus-discrete comparisons above, Figure 18 indicates that prediction difficulty on discrete designs is organized both by band (modal complexity) and by interface measure (geometric complexity), as expected when a spectral surrogate retains only a portion of the broadband Fourier content generated by discontinuities.

3.4. Band and Wavevector Encoding

For the FNO to ingest the conditioning constants of the eigenvalue PDE in equation 6, namely the wavevectors k_x, k_y and the band index b that selects which eigenmode to return, these scalars must be represented as matrices or tensors compatible with the existing channel wise input structure. Alternatives that inject scalars through specialized pathways would require heavy architectural alterations, which are hard to justify given that FNO theory suggests eigenvalue PDEs should be solvable with existing hyperparameter combinations. We therefore compared three strategies for encoding these constants as input fields: constant valued fields, spatial sinusoids, and the Gabor wavelet encoding described in Section 2.4.

The constant field representation is the naive default: k_x, k_y, and b are each broadcast to a constant $S \times S$ channel with $S = 32$. For the wavevectors, whose physical values lie in $[-\pi, \pi]$, these constants were normalized to the interval $[-1, 1]$ before broadcasting. Band indices $\{1, 2, 3, 4, 5, 6\}$ were likewise normalized to the

range $[0.1, 0.6]$. This representation fails by mode collapse, with the FNO outputting near constant fields regardless of conditioning inputs. The cause is spectral: the FFT of a constant field is zero everywhere except at the DC coefficient, so the frequency domain branch of each Fourier layer receives no information to propagate or train on past the first layer. The Fourier branch therefore cannot contribute to the prediction, leaving only the shallow spatial domain branch, which behaves like a shallow CNN and is unable to capture the complexity of the eigenvalue problem.

A spatial sinusoidal encoding restores unique and nondegenerate representations in the spectral domain and allows the FNO to learn the problem. On pixel coordinates $(x, y) \in \{0, \ldots, S-1\}$, wavevector and band were encoded as

$$I_k(x, y) = \cos\left(\frac{k_x}{S}x + \frac{k_y}{S}y\right), \qquad k_x, k_y \in [-\pi, \pi].$$

$$I_b(x, y) = \tfrac{1}{2}\left[\cos\left(\frac{2\pi b x}{S}\right) + \cos\left(\frac{2\pi b y}{S}\right)\right], \qquad b \in \{1, 2, 3, 4, 5, 6\}.$$

However, because a sinusoid is spectrally sparse, with energy concentrated at only a few wavenumbers, the Fourier branch remains weakly utilized, and predictions exhibit a persistent grainy, under-resolved structure relative to ground truth. Increasing model depth and width did not remove this graininess, indicating that the bottleneck was the encoding rather than model capacity.

The Gabor wavelet encoding resolves this by design. Wavelets are constructed to trade certainty between the spatial and spectral domains: neither representation is infinitely sharp, but both remain informationally rich. As a result, the conditioning inputs populate both branches of each Fourier layer with usable structure, both branches can train and contribute, and the model yields the accurate, well-resolved predictions reported above while enabling deterministic mode selection. These observations support the broader conclusion that input encodings should be matched to the spatial and spectral structure of the operator: encodings that populate only one domain underuse the FNO's Fourier branch, whereas wavelet encodings exercise both.

4. Conclusions

This work demonstrates that a single Fourier Neural Operator can learn multiple eigenmodes of the elastic wave equation across both continuous and discrete metamaterial geometries, recovering the eigenvectors as complex displacement fields and the eigenvalues as eigenfrequencies. For unseen geometries of both types, displacement predictions achieve relative errors typically between 10^{-2} and 10^{-4} and dispersion reconstructions average below 1% error, showing that one model can span two very different geometry distributions and output predictions of multiple orders of magnitude while allowing the user to select any eigenmode deterministically and repeatably through arbitrary choice of conditioning inputs. This capability crucially rests on how the conditioning constants are encoded. Constant field encodings collapse to a single DC coefficient in the spectral domain, preventing the Fourier branch of the FNO from propagating information and causing mode collapse. Sinusoidal encodings are able to train meaningfully but yield grainy, under-resolved predictions due to spectral sparsity. Gabor wavelet encodings, by contrast, trade decreasing certainty in the spatial and spectral representations to maintain informationally rich representations in both domains, allowing both branches of each Fourier layer to richly interact with the data, leading to superior learning speed and prediction performance.

The same spectral perspective explains the limits we observe. Prediction error grows with the interface length of discrete geometries and with band index, consistent with the truncated Fourier representation struggling to resolve the broadband content generated by sharp material interfaces and rapidly oscillating modes. Practitioners applying FNOs to interface-rich problems should therefore expect accuracy to track geometric discontinuity. Even so, the trained model is lightweight, roughly 2 GB at float16 precision, and evaluates in about 1 ms per sample on CPU, compared with about 1 s per sample for the corresponding MATLAB FEA solve on CPU. This makes the FNO a compelling surrogate for iterative design and analysis in metamaterials and other fields governed by eigenvalue PDEs. Future work includes generalization to other resolutions, physics-informed losses, and additional conditioning parameters such as material constants, expanding the accessible problem and solution space.

AI Usage Disclosure

Generative AI tools were used to assist with coding of experiments, translating MATLAB code to Python, and automating training pipelines. AI tools were also used for light editing of the manuscript. All scientific claims, results, and final wording were reviewed and approved by the authors.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Han Zhang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review and editing.
Alexander Ogren: Data curation, Software.
Cynthia Rudin: Supervision, Writing – review and editing.
Johann Guilleminot: Formal analysis, Methodology, Supervision, Writing – review and editing.
L. Catherine Brinson: Conceptualization, Funding acquisition, Methodology, Project administration, Resources, Software, Supervision, Visualization, Writing – review and editing.

Data availability

Python code and experiments can be found at `https://github.com/trutheresy/NO-2D-Metamaterials`.

MATLAB FEA code can be found at `https://github.com/aco8ogren/2D-dispersion.git`.

Acknowledgements

This work was supported by the NSERC Postgraduate Scholarship – Doctoral (PGSD-599307-2025) and the NSF AI for Understanding and Designing Materials Research Traineeship (DGE-2022040).

References

S1. Supplementary Information

S1.1. Model and Training Hyperparameters

Hyperparameter	**Candidate Settings**			
Fourier Layers	*4*	6	8	12
Hidden Channels	32	64	*128*	256
Activation Function	ReLU	*GELU*	tanh	Sigmoid
Loss Criterion	L1	L2	*Huber*	SSIM
Epochs	8	*12*	16	20
Learning Rate	10^{-4}	2×10^{-3}	10^{-2}	10^{-1}
Weight Decay	0	10^{-4}	10^{-3}	10^{-2}
Gamma	0	0.01	*0.1*	0.5
Step Size	0	1	*4*	10
Batch Size	64	128	256	*512*
Data Precision	F8 E4M3	F8 E5M2	*F16*	F32

Table S1: Combinations tried in grid search for optimal model and training hyperparameters.

S1.2. Loss Criterion

In the exploration of optimal training protocols for the FNO model, several loss functions were evaluated as training objectives: mean absolute error (MAE), mean squared error (MSE), normalized MAE, normalized MSE, Huber (smooth L1), and structural similarity index (SSIM). For comparison, validation performance was measured with MAE and MSE. Broadly speaking, models that performed best in MSE also performed best in MAE. The training objective that yielded the lowest MAE and MSE was Huber loss, which was used for all reported results. All comparisons used a learning rate of 2×10^{-3} with a per-epoch decay factor of 0.9. The mathematical definitions of each loss follow. Losses were evaluated and aggregated across all five output channels.

- The mean absolute error (MAE) loss is defined as

$$\mathcal{L}_{\text{MAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^{4} |\hat{u}_c(i,j) - u_c(i,j)|$$

Experimentally, models trained exclusively with MAE produced predictions with sharper interfaces and better-preserved boundary features than those trained solely with MSE. This behavior is consistent with the linear penalty of MAE, which does not disproportionately penalize isolated large residuals near discontinuities. Despite the improved qualitative sharpness, MAE training converged to higher validation MAE and MSE than Huber loss, indicating that the improved preservation of fine structural features came at the expense of overall numerical accuracy. This observation also motivated the hybrid training schedule in which optimization began with MAE before transitioning to MSE, combining the edge-preserving characteristics of MAE with the stronger refinement of large residuals provided by MSE.

- The mean squared error (MSE) loss is defined as

$$\mathcal{L}_{\text{MSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^{4} \left(\hat{u}_c(i,j) - u_c(i,j)\right)^2$$

  The quadratic penalty of MSE assigns increasingly large gradients to larger residuals, causing the optimizer to preferentially reduce regions with the greatest numerical error. As a consequence, the loss favors smooth predictions that minimize the overall energy of the residual rather than preserving sharp transitions. Experimentally, models trained exclusively with MSE produced outputs that reproduced the correct magnitude and large-scale structure of the solution, but exhibited diffuse interfaces and blurred boundaries. Fine-scale features were often replaced by low-amplitude, grainy artifacts, consistent with the tendency of MSE to average over uncertain or discontinuous regions in order to minimize the global squared error.

- A hybrid MAE–MSE training schedule was also evaluated, where the model was optimized using MAE for the first half of the training epochs before switching to MSE for the remaining half. This training strategy approximates the behavior of the Huber loss over the course of optimization. During the early stages of

training, prediction errors are typically large, making the linear penalty of MAE advantageous for preserving sharp interfaces while preventing large residuals from dominating the optimization. As training progresses and the average residual decreases, switching to MSE increases the emphasis on refining the remaining prediction errors and improving overall numerical accuracy.

Experimentally, this training schedule consistently outperformed training with either MAE or MSE alone for the same total number of epochs. The resulting predictions retained substantially sharper boundaries than models trained exclusively with MSE, while achieving lower validation MAE and MSE than models trained exclusively with MAE. Among the non-Huber training strategies investigated, the hybrid schedule produced the best overall balance between qualitative feature preservation and quantitative accuracy, converging to within approximately 10% of the validation error achieved by Huber loss. Although less adaptive than Huber, which transitions between MAE- and MSE-like behavior on a per-residual basis, the epoch-wise transition captures much of the same optimization benefit.

- The structural similarity index measure (SSIM) is defined as

$$\mathcal{L}_{\text{SSIM}} = 1 - \frac{1}{5}\sum_{c=0}^{4} \frac{(2\mu_{u_c}\mu_{\hat{u}_c} + C_1)\,(2\sigma_{u_c\hat{u}_c} + C_2)}{\left(\mu_{u_c}^2 + \mu_{\hat{u}_c}^2 + C_1\right)\left(\sigma_{u_c}^2 + \sigma_{\hat{u}_c}^2 + C_2\right)}, \quad \text{where}$$

$$\mu_{u_c} = \frac{1}{HW}\sum_{i=0}^{H-1}\sum_{j=0}^{W-1} u_c(i,j), \quad \sigma_{u_c\hat{u}_c} = \frac{1}{HW}\sum_{i=0}^{H-1}\sum_{j=0}^{W-1} (u_c(i,j) - \mu_{u_c})\,(\hat{u}_c(i,j) - \mu_{\hat{u}_c})$$

A significant limitation of SSIM arises when it is applied to signed-valued fields. Unlike its original application to nonnegative image intensities, a perfect sign-reversed field ($\hat{u} = -u$) can achieve nearly the same SSIM score as a perfect reconstruction ($\hat{u} = u$). Consequently, a model trained solely with SSIM may converge to physically incorrect solutions containing polarity inversions while still minimizing the loss. The following derivation demonstrates the mathematical origin of this ambiguity.

For the purposes of the following discussion, we consider the common case where the stabilizing constants satisfy

$$C_1 \ll 2\mu_{u_c}^2, \qquad C_2 \ll 2\sigma_{u_c}^2,$$

allowing them to be neglected.

For a correct reconstruction,

$$u_c^{(+)} = u_c,$$

the SSIM score is

$$\mathrm{SSIM}(u_c, u_c^{(+)}) = \frac{\left(2\mu_{u_c}^2\right)\left(2\sigma_{u_c}^2\right)}{\left(2\mu_{u_c}^2\right)\left(2\sigma_{u_c}^2\right)} = 1.$$

Now consider a sign-reversed reconstruction. The field and its mean reverse sign,

$$u_c^{(-)} = -u_c, \qquad \mu_{u_c^{(-)}} = -\mu_{u_c},$$

while the covariance reverses sign and the variance remains unchanged,

$$\sigma_{u_c, u_c^{(-)}} = -\sigma_{u_c, u_c}, \qquad \sigma_{u_c^{(-)}}^2 = \sigma_{u_c}^2.$$

Substituting these relationships into the SSIM expression gives

$$\mathrm{SSIM}(u_c, u_c^{(-)}) = \frac{\left(-2\mu_{u_c}^2\right)\left(-2\sigma_{u_c}^2\right)}{\left(2\mu_{u_c}^2\right)\left(2\sigma_{u_c}^2\right)} = 1.$$

The numerator therefore differs from that of a correct reconstruction by two factors of -1, and thus, under these conditions, SSIM assigns essentially the same similarity score to a field and its sign-reversed counterpart as it does to a perfect reconstruction. This ambiguity constitutes a significant limitation when SSIM is applied to signed-valued regression problems where the polarity of the solution is physically meaningful. During training, the network frequently converged to predictions in which localized regions or the entire image were

the negative of the correct solution while still achieving a favorable SSIM objective. Although the resulting fields exhibited accurate interface locations, boundary geometry, and overall morphology, the polarity inversions rendered the predictions physically incorrect. Consequently, SSIM should not be used as the sole training criterion for signed-valued fields unless it is supplemented by an additional loss that explicitly penalizes sign reversals.

- The normalized mean absolute error (NMAE) loss is defined as

$$\mathcal{L}_{\text{NMAE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^{4} \frac{|\hat{u}_c(i,j) - u_c(i,j)|}{|u_c(i,j)| + \varepsilon}$$

  Unlike MAE, NMAE weights each prediction error by the inverse magnitude of the target value, causing the optimization to place greater emphasis on regions where the ground-truth solution is small in scale. Consequently, the loss seeks to minimize relative error rather than absolute error, preventing low-magnitude features from being dominated by high-magnitude regions during training.

  Experimentally, NMAE successfully reduced the relative error in low-amplitude portions of the solution. However, this improvement came at the expense of increased overall MAE and MSE compared with training using the standard MAE loss. For the present application, the absolute magnitude of the prediction error was determined to be of greater practical importance than the relative error. As a result, the improved accuracy in low-valued regions did not outweigh the degradation in overall numerical accuracy, making NMAE a less suitable optimization criterion for the objectives of this work. In other applications, however, NMAE can be a valid and sometimes preferable alternative to standard MAE.

- The normalized mean squared error (NMSE) loss is defined as

$$\mathcal{L}_{\text{NMSE}} = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{c=0}^{4} \frac{\left(\hat{u}_c(i,j) - u_c(i,j)\right)^2}{\left(u_c(i,j)\right)^2 + \varepsilon}$$

Similar to NMAE, NMSE normalizes the prediction error by the magnitude of the target field, causing the optimization to prioritize relative error rather than absolute error. The quadratic penalty retains the characteristic behavior of MSE by assigning greater weight to larger residuals, while the normalization emphasizes regions where the target magnitude is small.

Experimentally, NMSE reduced the relative error in low-amplitude regions compared with standard MSE. However, this improvement was accompanied by increased overall validation MAE and MSE, performing worse than conventional MSE optimization. As with NMAE, the objective of this work was to minimize the absolute prediction error rather than the relative error. Consequently, the increased emphasis placed on low-magnitude regions did not translate into improved overall predictive performance, making NMSE less appropriate for the present application.

S1.3. Algorithms for Input Wavelet Encoding

The band and wavevector conditioning channels used by the FNO are generated by the Gabor wavelet embeddings below, which match the implementations in the accompanying code.

Algorithm 1 1D Gabor Wavelet Embedding from Band Index

INPUT: Band index b; grid size $S \leftarrow 32$; frequency scale $r \leftarrow 2.0$

Create linspace vectors $x, y \in [-1, 1]$ with S points

Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$

Compute base frequency:

$$f \leftarrow (1.0 + |b|) \cdot \frac{S}{8}$$

Compute rotation angle:

$$\theta \leftarrow (b \bmod 8) \cdot \frac{S}{16}$$

Rotate coordinates:

$$X_\theta \leftarrow X \cos\theta + Y \sin\theta, \quad Y_\theta \leftarrow -X \sin\theta + Y \cos\theta$$

Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.3}{r}$$

Compute Gabor wavelet embedding:

$$\psi_b(x, y) \leftarrow \exp\left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2}\right) \cdot \cos(f \cdot X_\theta)$$

RETURN: $\psi_b \in \mathbb{R}^{S \times S}$

In Algorithm 1, the grid size $S = 32$ matches the geometry resolution used by the FNO. For this resolution the base frequency is $f = (1 + |b|)\, S/8$, so successive bands increase both carrier frequency and orientation. The modulo in θ repeats orientation every eight bands, while f continues to increase, so embeddings remain distinguishable beyond eight bands. The constants $r = 2.0$, $S/16$, and 0.3 were chosen so that the wavelets occupy most of the spatial domain and remain visually distinct across bands.

Algorithm 2 2D Gabor Wavelet Embedding for Wavevectors

INPUT: Wavevector components k_x, k_y (radians); grid size $S \leftarrow 32$; frequency scale $r \leftarrow 1.0$

Set constants $f_0 \leftarrow 2.20$, $\alpha \leftarrow 41$, $N_x \leftarrow 13$, $N_y \leftarrow 25$

Create linspace vectors $x, y \in [-1, 1]$ with S points

Construct meshgrid $X, Y \leftarrow \text{meshgrid}(x, y)$

Compute frequencies:

$$f_x \leftarrow (f_0 + k_x)\,\alpha, \quad f_y \leftarrow (f_0 + k_y)\,\alpha$$

Compute rotation angles:

$$\theta_x \leftarrow (k_x \bmod N_x) \cdot \frac{\pi}{N_x}, \quad \theta_y \leftarrow (k_y \bmod N_y) \cdot \frac{\pi}{N_y}$$

Rotate coordinates:

$$X_\theta \leftarrow X \cos\theta_x + Y \sin\theta_y, \quad Y_\theta \leftarrow -X \sin\theta_x + Y \cos\theta_y$$

Set Gaussian envelope widths:

$$\sigma_x \leftarrow \sigma_y \leftarrow \frac{0.5}{r}$$

Compute Gabor wavelet embedding:

$$\psi_{k_x,k_y}(x, y) \leftarrow \exp\left(-\frac{X_\theta^2}{2\sigma_x^2} - \frac{Y_\theta^2}{2\sigma_y^2}\right) \cdot \sin(f_x X_\theta) \cdot \sin(f_y Y_\theta)$$

RETURN: $\psi_{k_x,k_y} \in \mathbb{R}^{S \times S}$

In Algorithm 2, k_x and k_y enter as continuous radian valued wavevector components. Distinct modulo periods $N_x = 13$ and $N_y = 25$ reduce aliasing under interchange of the two components. The offset f_0 and scale α map the physical range of k into carrier frequencies that remain well resolved on the $S \times S$ grid, while

$\sigma = 0.5/r$ sets the envelope width so that the patterns remain spatially localized yet informative in both domains.

S1.4. Wavelet Decoding

For purposes of checking general applicability, an experiment was run to check if a positive scalar may be embedded in a Gabor wavelet image and recovered after pixelization, storage, or inference. Algorithms 3 and 4 implement this encode–decode pair by spectral peak detection with centroid refinement. In the experiment, we test recovery over $[1, 8000]$, spanning about four orders of magnitude. Despite discretization onto a finite $S \times S$ grid and limited numeric precision associated with float16 storage and arithmetic, the decoded values remain accurate across the full range. Figures S1 and S2 show representative encodings and the corresponding round trip error.

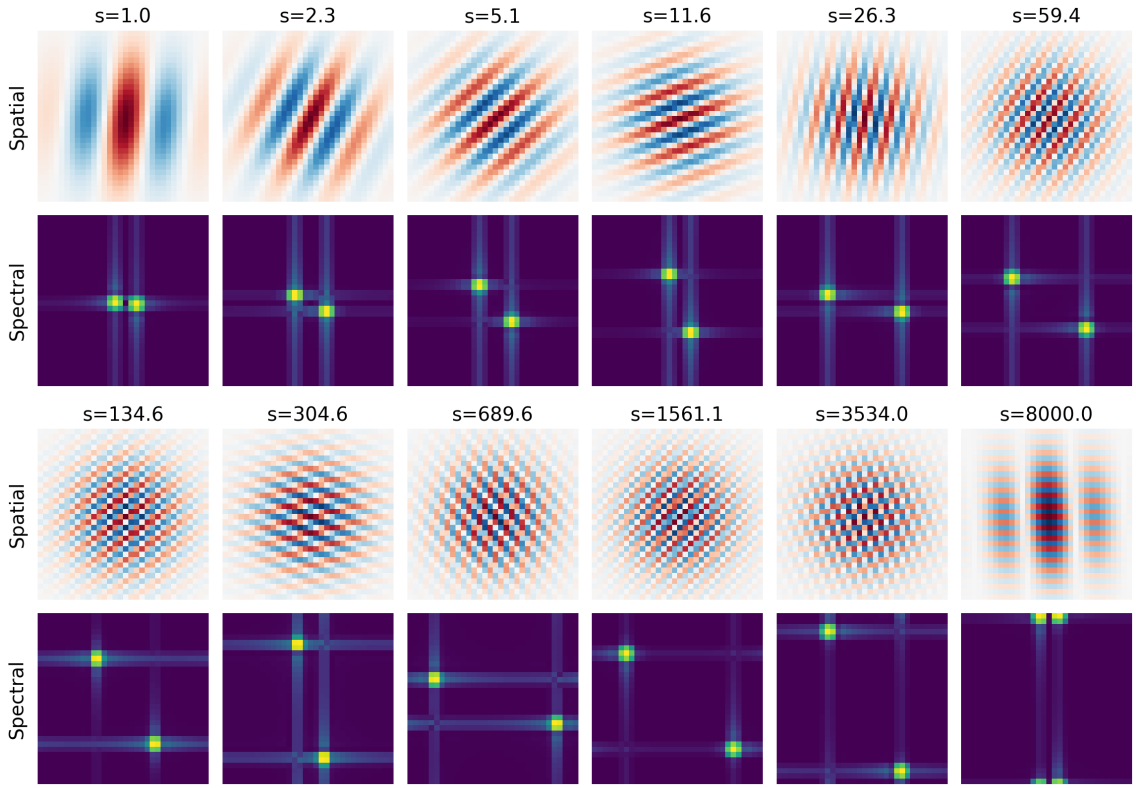


Figure S1: Log spaced Gabor wavelet encodings of scalars from 1 to 8000. Alternating rows show the spatial wavelet images and the corresponding Fourier magnitude spectra.

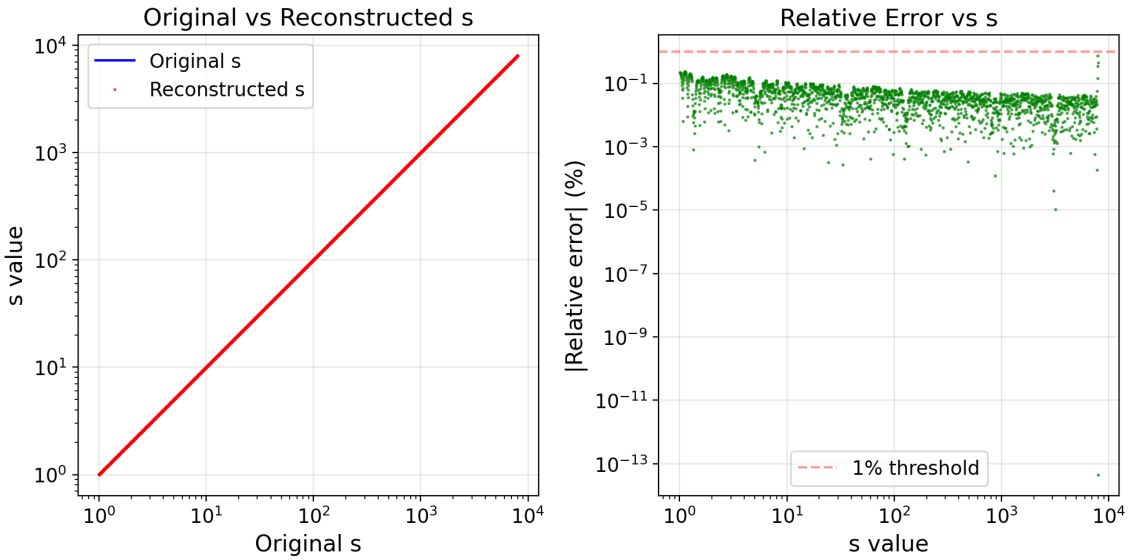


Figure S2: Encode–decode error for scalars from a minimum of 1 to a maximum of 8000.

Algorithm 3 Scalar Wavelet Encoding

1: **INPUT:** Scalar $s > 0$; image size $S \leftarrow 32$; bounds $[s_{\min}, s_{\max}]$; spectral-index bounds $[k_{\min}, k_{\max}]$

2: Set constants $\gamma \leftarrow 1,\ \phi \leftarrow 0,\ \sigma \leftarrow 8,\ \theta_{\min} \leftarrow \pi/30,\ \theta_{\max} \leftarrow \pi/2 - \pi/30$

3: Compute log-domain values:

$$\ell_s \leftarrow \log s, \quad \ell_{\min} \leftarrow \log s_{\min}, \quad \ell_{\max} \leftarrow \log s_{\max}$$

4: Compute normalized position and spectral index:

$$t \leftarrow \mathrm{clip}\left(\frac{\ell_s - \ell_{\min}}{\ell_{\max} - \ell_{\min}}, 0, 1\right), \quad k \leftarrow k_{\min} + t\,(k_{\max} - k_{\min})$$

5: Compute log width per unit k:

$$\Delta_\ell \leftarrow \frac{\ell_{\max} - \ell_{\min}}{k_{\max} - k_{\min}}$$

6: Compute orientation from within-band log position:

$$\theta \leftarrow \theta_{\min} + \left(\frac{(\ell_s - \ell_{\min}) \bmod \Delta_\ell}{\Delta_\ell}\right)(\theta_{\max} - \theta_{\min})$$

7: Build centered grid $X, Y \in \{-S/2, \ldots, S/2 - 1\}$, then rotate:

$$X_\theta \leftarrow X\cos\theta + Y\sin\theta, \quad Y_\theta \leftarrow -X\sin\theta + Y\cos\theta$$

8: Set envelope widths $\sigma_x \leftarrow \sigma,\ \sigma_y \leftarrow \gamma\sigma_x$, and carrier frequency $\omega \leftarrow 2\pi k/S$

9: Compute wavelet image:

$$\psi_s \leftarrow \exp\left(-\frac{1}{2}\left(\frac{X_\theta^2}{\sigma_x^2} + \frac{Y_\theta^2}{\sigma_y^2}\right)\right)\cos(\omega X_\theta + \phi)$$

10: **RETURN:** $\psi_s \in \mathbb{R}^{S\times S},\ k,\ \theta$

Algorithm 4 Scalar Wavelet Decoding

INPUT: Wavelet image $\psi \in \mathbb{R}^{S\times S}$; bounds $[s_{\min}, s_{\max}]$, $[k_{\min}, k_{\max}]$, $[\theta_{\min}, \theta_{\max}]$

Set constants for centroid refinement: radius $R \leftarrow 3$, power $p \leftarrow 2$

Mean-center image $\psi_c \leftarrow \psi - \text{mean}(\psi)$, then compute the centered 2D discrete Fourier magnitude:

$$\hat{\psi}(k_x, k_y) \leftarrow \sum_{m=0}^{S-1}\sum_{n=0}^{S-1} \psi_c(m,n)\, \exp\left(-2\pi i\left(\frac{k_x m}{S} + \frac{k_y n}{S}\right)\right), \quad F(k_x, k_y) \leftarrow |\hat{\psi}(k_x, k_y)|$$

Keep upper-half-plane spectral support (plus positive-frequency center row) and find dominant peak index (r^*, c^*)

Define symmetric peak partner about center, then compute weighted local centroid around (r^*, c^*) within radius R, using weights $w = F^p$ and excluding points closer to the symmetric partner

Convert refined centroid to wavevector coordinates (k_x, k_y), then:

$$k_{\text{ext}} \leftarrow \sqrt{k_x^2 + k_y^2}, \quad \theta_{\text{ext}} \leftarrow \text{atan2}(k_y, k_x) \bmod \pi$$

Map extracted k to approximate log value:

$$\ell_{\text{approx}} \leftarrow \ell_{\min} + \text{clip}\left(\frac{k_{\text{ext}} - k_{\min}}{k_{\max} - k_{\min}}, 0, 1\right)\, (\ell_{\max} - \ell_{\min})$$

Let $\Delta_\ell \leftarrow (\ell_{\max} - \ell_{\min})/(k_{\max} - k_{\min})$, compute within-band position from angle:

$$\alpha \leftarrow \text{clip}\left(\frac{\theta_{\text{ext}} - \theta_{\min}}{\theta_{\max} - \theta_{\min}}, 0, 1\right), \quad \delta_\ell \leftarrow \alpha\, \Delta_\ell$$

Find nearest consistent log value by testing neighboring bands $b-1, b, b+1$:

$$\ell^\star \leftarrow \arg\min_{\ell \in \{\ell_{\min} + j\Delta_\ell + \delta_\ell\}} |\ell - \ell_{\text{approx}}|$$

Clip $\ell^\star$ to $[\ell_{\min}, \ell_{\max}]$, then decode:

$$s_{\text{ext}} \leftarrow \exp(\ell^\star)$$

RETURN: $s_{\text{ext}},\ k_{\text{ext}},\ \theta_{\text{ext}}$

Algorithms 3 and 4 summarize the forward and inverse mapping used in this experiment. The forward mapping places the scalar on a log scale and jointly encodes magnitude and orientation into a Gabor-like pattern. The inverse mapping recovers an estimate by spectral peak detection with centroid refinement, followed by a consistency step that combines extracted magnitude and angle to reconstruct the most likely log value.